



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CURSO DE ÉTICA Y SEGURIDAD DE LOS DATOS

# Sistema de Votación Digital Segura

Informe Final: Implementación, Ética y Retrospectiva

## Integrantes:

Jorge Alexander Leon Villareyes

Diva Stewart Maquera Bobadilla

Rodrigo Li Chumpitaz

Camila Pamela Acosta Arostegui

## Profesor:

Jose Carlos Pazos Ortiz

29 de noviembre de 2025

# Índice

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1. Introducción y Motivación</b>                       | <b>2</b> |
| <b>2. Entrega 1: Fundamentos y Arquitectura Base</b>      | <b>3</b> |
| 2.1. Diseño y Motivación Inicial . . . . .                | 3        |
| 2.2. Arquitectura Técnica . . . . .                       | 3        |
| 2.3. Criptografía Implementada . . . . .                  | 3        |
| <b>3. Entrega 2: Evolución, Seguridad y UX</b>            | <b>4</b> |
| 3.1. Implementación Propuesta vs. Lograda . . . . .       | 4        |
| 3.2. Detalles Técnicos de la Mejora . . . . .             | 4        |
| 3.2.1. Integridad Avanzada: Proof of Work (PoW) . . . . . | 4        |
| 3.2.2. Anonimato: Ballot Tokens . . . . .                 | 4        |
| 3.2.3. Definición y Segregación de Roles . . . . .        | 5        |
| <b>4. Privacidad y Ética de Datos</b>                     | <b>6</b> |
| 4.1. Minimización de Datos . . . . .                      | 6        |
| 4.2. Derecho al Anonimato . . . . .                       | 6        |
| 4.3. Transparencia y Auditabilidad . . . . .              | 6        |
| <b>5. Lecciones Aprendidas y Retrospectiva</b>            | <b>7</b> |
| 5.1. Lecciones Aprendidas . . . . .                       | 7        |
| 5.2. Retrospectiva del Equipo . . . . .                   | 7        |
| <b>6. Conclusiones</b>                                    | <b>7</b> |

# 1. Introducción y Motivación

El presente informe documenta de manera exhaustiva el ciclo de desarrollo del "Sistema de Votación Digital Segura". Este proyecto nace de la motivación de abordar la problemática de la confianza en los sistemas electorales electrónicos.

El objetivo principal es diseñar e implementar una solución que mitigue riesgos críticos como la suplantación de identidad, la manipulación de resultados y la violación del secreto del voto, utilizando principios de criptografía moderna y ética de datos. El código fuente y este informe se encuentran alojados públicamente para garantizar transparencia en:

[https://github.com/JorgeL2005/ESD\\_Project](https://github.com/JorgeL2005/ESD_Project)

## 2. Entrega 1: Fundamentos y Arquitectura Base

### 2.1. Diseño y Motivación Inicial

La primera fase se centró en establecer una infraestructura funcional que permitiera el flujo básico: Registro  $\rightarrow$  Autenticación  $\rightarrow$  Voto Cifrado  $\rightarrow$  Auditoría. La decisión de diseño fundamental fue evitar el almacenamiento de votos en texto plano. Se optó por un esquema de cifrado asimétrico donde el sistema posee una clave pública para cifrar y una privada (resguardada por el administrador) para el escrutinio final.

### 2.2. Arquitectura Técnica

**Backend Framework:** FastAPI (Python) por su velocidad y tipado estático.

**Base de Datos:** SQLite gestionada con SQLAlchemy.

**Frontend:** HTML5/JS estático servido por el backend.

### 2.3. Criptografía Implementada

- **Confidencialidad (RSA-OAEP):** Cifrado con la Clave Pública del Sistema.
- **Autenticidad (RSA-PSS):** Firma digital con la clave privada del usuario.
- **Integridad (Hash Chain):** Encadenamiento de votos mediante SHA-256 (`prev_hash_hex`).

## 3. Entrega 2: Evolución, Seguridad y UX

Esta sección detalla las mejoras implementadas tras el análisis de la primera entrega, enfocándose en robustez y privacidad.

### 3.1. Implementación Propuesta vs. Lograda

| Implementación Propuesta              | Implementación Lograda                                                                                                                       |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticación básica con hash simple. | Autenticación robusta usando <b>PBKDF2-SHA256</b> con 310,000 iteraciones para resistir fuerza bruta.                                        |
| Registro abierto de usuarios.         | <b>Registro Administrado:</b> Solo el rol admin puede crear usuarios para prevenir ataques Sybil.                                            |
| Ledger simple (Hash Chain).           | <b>Proof of Work (PoW):</b> Implementación de minado ( <b>nonce</b> ) para escribir en el ledger, dificultando la reescritura del historial. |
| Voto vinculado al usuario.            | <b>Ballot Tokens:</b> Desacoplamiento total mediante tokens de un solo uso que separan la identidad del voto almacenado.                     |

### 3.2. Detalles Técnicos de la Mejora

#### 3.2.1. Integridad Avanzada: Proof of Work (PoW)

Se implementó un mecanismo de consenso en `crypto_utils.py`. La función `submit_vote` ahora resuelve el desafío:

$$\text{SHA256}(\text{Cifrado} + \text{PrevHash} + \text{Nonce}) < \text{Target}$$

Esto añade un costo computacional a la escritura, mitigando spam y manipulación masiva.

#### 3.2.2. Anonimato: Ballot Tokens

Protocolo diseñado para la ética de datos:

1. Usuario autenticado solicita token (`/auth/issue-ballot`).
2. Sistema emite JWT temporal con `jti` único.
3. Voto se envía con `X-Ballot-Token`, sin ID de usuario en la base de datos de votos.

### 3.2.3. Definición y Segregación de Roles

Para garantizar la separación de funciones crítica en un proceso electoral, el sistema implementa un control de acceso estricto basado en tres roles diferenciados:

**Administrador (Autoridad Electoral):** Es el único rol autorizado para gestionar el alta de usuarios. Este proceso se define explícitamente como **estrictamente presencial**: el administrador debe constatar la identidad física del ciudadano antes de generar sus credenciales. Como parte de este protocolo de seguridad, el administrador entrega en mano al usuario el archivo `.txt` conteniendo su clave privada. Finalmente, es el responsable de activar el descifrado de los resultados.

**Votante:** Tras haber validado su identidad presencialmente y recibido su usuario, contraseña y clave privada, el votante adquiere la capacidad de **emitir su voto desde cualquier ubicación**. Su interacción con el sistema consiste en autenticarse, solicitar el token de anonimato y enviar su voto cifrado y firmado, garantizando así la participación remota segura.

**Auditor:** Rol de supervisión diseñado para observadores externos. Tiene acceso de lectura al Ledger y a las herramientas de verificación matemática (`/audit/verify-ledger`) para certificar que la cadena de bloques y el Proof of Work son válidos. Adicionalmente, cuenta con acceso a los resultados agregados, lo que le permite visualizar el conteo de votos y monitorear el desarrollo de la elección (tendencia de votación) en tiempo real.

## 4. Privacidad y Ética de Datos

El diseño del sistema se adhiere a principios éticos fundamentales para proteger los derechos digitales de los votantes.

### 4.1. Minimización de Datos

El sistema recolecta únicamente la información estrictamente necesaria. En la tabla `Vote`, no existe una columna `user_id`. La vinculación entre el votante y el acto de votar se rompe intencionalmente tras la validación del *Ballot Token*, asegurando que ni siquiera un administrador de base de datos pueda correlacionar un voto específico con una persona.

### 4.2. Derecho al Anonimato

A diferencia de los sistemas bancarios donde la trazabilidad es obligatoria, en un sistema electoral el anonimato es un requisito funcional. El uso de cifrado asimétrico garantiza que el contenido del voto permanezca oculto durante todo el proceso de recolección, y solo sea revelado en la fase de escrutinio bajo supervisión administrativa.

### 4.3. Transparencia y Auditabilidad

El código es de código abierto (*Open Source*), permitiendo que cualquier ciudadano o entidad audite los algoritmos criptográficos utilizados. Además, la implementación del Ledger público (endpoint `/ledger`) permite a los auditores verificar matemáticamente que los votos no han sido alterados, sin necesidad de ver su contenido en claro.

## 5. Lecciones Aprendidas y Retrospectiva

### 5.1. Lecciones Aprendidas

1. **Compatibilidad Criptográfica:** Inicialmente se utilizó `bcrypt`, pero se encontraron limitaciones con su longitud máxima de entrada (72 bytes). Aprendimos la importancia de evaluar las primitivas criptográficas (migrando a `PBKDF2`) según los requisitos del sistema y no solo por popularidad.
2. **Gestión de Claves (UX vs. Seguridad):** Entregar la responsabilidad de la clave privada al usuario final (Primary Key) es seguro teóricamente, pero presenta enormes desafíos de usabilidad. Muchos usuarios pierden el archivo o no entienden el formato PEM. Se implementaron ayudas en el frontend (carga de archivos, validación de cabeceras) para mitigar esto.
3. **Limitaciones de Python para PoW:** Implementar Proof of Work en un lenguaje interpretado como Python tiene penalizaciones de rendimiento significativas comparado con C++ o Rust. Para un prototipo académico es funcional, pero para producción requeriría optimización a bajo nivel.

### 5.2. Retrospectiva del Equipo

- **Lo que funcionó bien:** La separación clara entre Frontend y Backend permitió trabajar en paralelo, optimizando el tiempo de desarrollo. Asimismo, la adopción temprana de FastAPI facilitó la documentación automática y la estandarización de la API.
- **Áreas de mejora técnica:** La validación de firmas en el cliente (Frontend) usando *WebCrypto API* presentó dificultades de integración con el formato PEM generado por Python. Futuras iteraciones deberían estandarizar el uso de JSON Web Keys (JWK) para mejorar la interoperabilidad.
- **Gestión de versiones y flujo de trabajo:** Se identificó una deficiencia en la estrategia de *commits*. Los integrantes tendimos a realizar avances extensos en ramas aisladas sin realizar confirmaciones ni integraciones frecuentes, lo que incrementó considerablemente la complejidad y cantidad de conflictos de fusión (*merge conflicts*) al momento de unificar el código.

## 6. Conclusiones

El *Sistema de Votación Digital Segura* ha logrado cumplir con sus objetivos de diseño, proporcionando una plataforma que equilibra la seguridad técnica con la ética de datos. La evolución hacia la Entrega 2, con la incorporación de *Ballot Tokens* y *Proof of Work*, demuestra la capacidad del sistema para resistir ataques modernos mientras protege la identidad del ciudadano.